

Silence of the RAM:

Constant-Memory Safety Probing and the Frontiers of Streaming Guardrails

Shreyas Venkata Ramanan
shreyasvramanan.2545@gmail.com

Abstract

Activation probes are now deployed safety infrastructure, shipped inside frontier assistants and used as white-box members of untrusted-monitor ensembles. They are also memory-bound: an attention-pooled probe materialises a score tensor over the whole sequence, so its footprint grows with the one quantity an adversary controls, the context length. We study aggregation as the independent variable, holding the token transform fixed, and characterise a hard-maximum probe (*MultiMax*) against softmax attention pooling, mean pooling and full self-attention across $N \in [128, 131,072]$. Claims are established both analytically and empirically on *real* residual streams read out of an open-weight model (Mistral-7B-v0.1, layers 16, 24 and 31), not on synthetic activations alone. Three results follow.

(i) **Systems.** MultiMax streams its reduction and carries only per-head state, giving activation overhead of $\Theta(\min(C, N))$ in the chunk width C : measured at a flat 19.1 MiB from $N=8,192$ to 131,072 ($\alpha=0.000$) against 652.1 MiB for softmax pooling ($\alpha=0.842$), while self-attention reaches 10.1 GiB and exhausts memory. A chunk ablation confirms the constant is chosen, not free. On real Mistral-7B activations at $d=4096$ the same law holds: overhead is flat at 8.0 MiB from $N=4096$ to 131,072 against 641.0 MiB for softmax pooling. Under a needle-disjoint split MultiMax also leads detection, but only narrowly (mean AUROC 0.731 against 0.690 and 0.696), and it collapses to chance at the final layer.

(ii) **Optimisation.** The hard-max subgradient is supported on H tokens per sequence, and at weak signal this prevents learning entirely (0.00 recall at its own training length). Annealing a *normalised* Boltzmann operator $\tau \log(\frac{1}{N} \sum_j e^{s_j/\tau})$ to $\tau \rightarrow 0$ restores 1.00 recall; the normalisation is load-bearing, since plain LogSumExp injects a length-dependent $H\tau \log N$ offset. Calibration by Platt scaling reduces Brier from 0.0538 to 0.0122 and makes uncertainty gating usable at all.

(iii) **Limits.** Against an adversary who fragments a fixed signal budget across m non-contiguous tokens, MultiMax and softmax pooling fail together at $m=64$; past that boundary softmax retains substantially better ranking (AUROC 0.716 vs 0.523 at $m=256$). We therefore do not claim a general detection advantage. The defensible claim is architectural: MultiMax is the right $\Theta(1)$ -memory *first stage* of a cascade, not a replacement for inspection.

1 Introduction

Monitoring a frontier model’s activations for misuse is no longer a research convenience. Probes are deployed in user-facing systems [15] and appear as white-box members inside AI-control protocols, where a cheap monitor gates an untrusted policy [9, 11]. Two properties then matter simultaneously, and they are usually studied apart: the probe must *detect*, and it must do so within a memory budget that does not grow with the context an adversary supplies.

That second requirement is sharp. Context windows now reach 10^5 – 10^6 tokens, and a guardrail that allocates memory proportional to the input is a guardrail whose cost the attacker sets. Kramár et al. [15] report that production probes fail under short-to-long distribution shift and that architecture choice, not merely training data, governs the outcome. Chen et al. [7] trace long-context guardrail failure to attention dilution. Neither isolates aggregation as a controlled variable.

We do. Fixing a shared token transform ϕ , we vary only the sequence reduction ρ , and compare four aggregators over five orders of magnitude in N . Our contributions:

- **A streaming hard-max probe with a measured memory law.** MultiMax carries (B, H) state across chunks, giving $\Theta(\min(C, N))$ activation overhead. We verify this with a chunk $\times$ length ablation rather than asserting $\mathcal{O}(1)$ (§5.3, Rem. 5.1).
- **An optimisation obstruction and its fix.** We identify the subgradient support of the maximum as the reason hard-max probes can fail to train at weak signal, and resolve it with normalised Boltzmann annealing (§3).
- **An honest boundary.** We construct the adversary that specifically attacks a maximum, locate the failure at $m=64$, and report that softmax pooling degrades more gracefully past it. This bounds our own claim (§6.1).
- **A calibrated cascade with a real second stage.** Sum-of-maxima logits are upward-biased; we show temperature scaling is structurally insufficient and that budget-driven thresholding is the correct interface (§4.3), then instantiate stage 2 with a deployed guardrail model rather than a stand-in (§6.3).
- **Validation on real residual streams.** Every detection claim is repeated on activations read out of Mistral-7B-v0.1 at layers 16, 24 and 31. The memory law and the aggregator ordering both survive; the hard maximum additionally collapses at the final layer, which is a limit the synthetic setting could not have exposed (§5.1).
- **Hyperparameters chosen rather than asserted.** A $c \times H$ grid shows the usable region is a staircase, and a direct measurement of pre-clamp logits identifies saturation of both classes, not saturation as such, as the failure mode (§5.6).

All numbers below are read programmatically from a single benchmark artifact; a claims checker verifies every figure quoted in this paper against it.

2 Aggregation as the Independent Variable

Let $x_{i,j} \in \mathbb{R}^d$ be the residual-stream activation at layer ℓ and position $j \in \{1, \dots, N\}$. A probe is

$$f_\theta(X_i) = \rho(\phi(x_{i,1}), \dots, \phi(x_{i,N})), \quad y_{i,j} = \phi(x_{i,j}) = \sigma(W_1 x_{i,j} + b_1) \in \mathbb{R}^m. \quad (1)$$

All probes share ϕ ; only ρ differs. Writing $\alpha_j \geq 0$, $\sum_j \alpha_j = 1$ for a convex pooling weight, mean pooling sets $\alpha_j = 1/N$ and softmax attention [24] sets $\alpha_j \propto \exp(q^\top y_j / \sqrt{m})$, both giving logit $w^\top (\sum_j \alpha_j y_j) + b$. MultiMax is not of this form:

$$a_h(X_i) = \max_{1 \leq j \leq N} v_h^\top y_{i,j}, \quad \text{logit}_i = \sum_{h=1}^H a_h(X_i) + b, \quad (2)$$

with predictions through a bounded link $\hat{p} = \varsigma(\Pi_{[-c,c]}(\text{logit}))$, $\varsigma(z) = (1 + e^{-z})^{-1}$, $c = 10$.

2.1 The bounded link and gradient propagation under saturation

The clamp exists for a numerical reason. A MultiMax logit is a sum of H maxima, and by Remark 2.6 each summand grows like $\tau\sqrt{2\log N}$ even under the null, so $|z|$ drifts upward with context length. In fp16 the sigmoid saturates to exactly 0 or 1 well before that, and the binary cross-entropy then returns a non-finite loss. Bounding z removes the overflow. It also, if implemented naively, removes the gradient.

Remark 2.1 (The clamp must not become the trap it prevents). $\Pi_{[-c,c]}$ has derivative $\mathbb{K}[|z| < c]$, identically zero on the saturated region, so with $\mathcal{L}$ the loss and θ any parameter,

$$\frac{\partial \mathcal{L}}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{p}} \varsigma'(\Pi(z))}_{\text{finite}} \cdot \underbrace{\mathbb{K}[|z| < c]}_{=0 \text{ when } |z| > c} \cdot \frac{\partial z}{\partial \theta} = 0 \quad \text{whenever } |z| > c. \quad (3)$$

A guard introduced to avoid vanishing gradients *creates* them, and does so precisely in the regime it was added for. We measured exactly 0.000 gradient norm on a saturated logit. Substituting the straight-through

estimator [3],

$$\tilde{\Pi}(z) = z + \text{sg}(\Pi_{[-c,c]}(z) - z), \quad \tilde{\Pi}(z) = \Pi_{[-c,c]}(z) \quad (\text{forward}), \quad \frac{\partial \tilde{\Pi}}{\partial z} \equiv 1 \quad (\text{backward}), \quad (4)$$

restores (3) to $\partial \mathcal{L} / \partial \theta = (\partial \mathcal{L} / \partial \hat{p}) \zeta'(\Pi(z)) \partial z / \partial \theta$. The surviving factor $\zeta'(\Pi(z)) = \zeta(c)(1 - \zeta(c)) \approx 4.54 \times 10^{-5}$ at $c = 10$ is small but strictly positive, so a saturated unit is slowed rather than killed. At $N=131,072$ with a raw logit of 46.75 clamped to 10, parameter gradients measure $\sum_{\theta} |\nabla_{\theta}| = 2.81 \times 10^5$; with the plain clamp the same configuration gives exactly 0.000.

We note the estimator is biased: forward and backward correspond to different functions, so $\tilde{\Pi}$ is not the gradient of any scalar. This is acceptable here because the bias is confined to a region the model should leave, since the update always points toward $|z| < c$, where forward and backward agree again and the estimator becomes exact.

2.2 Dilution

Assumption 2.2 (Localised needle). A length- N sequence contains an attack segment $\mathcal{A}$, $|\mathcal{A}| = k$ fixed, with $v_h^\top y_j = \mu_a + \varepsilon_j$ for $j \in \mathcal{A}$ and $\mu_b + \varepsilon_j$ otherwise, ε_j i.i.d. zero-mean sub-Gaussian with proxy τ^2 , and $\Delta = \mu_a - \mu_b > 0$.

Proposition 2.3 (Mean pooling dilutes as $\Theta(1/N)$). *Under Assumption 2.2 the mean-pooled shift is $k\Delta/N$ with fluctuation $\tau/\sqrt{N}$, so $\text{SNR}_{\text{mean}}(N) = (k\Delta/\tau) N^{-1/2} \rightarrow 0$.*

Proof. Linearity gives $(k\mu_a + (N - k)\mu_b)/N - \mu_b = k\Delta/N$; independence gives variance τ^2/N . $\square$

Proposition 2.4 (Softmax mass is $\mathcal{O}(1/N)$ for bounded logit gap). *With $\gamma = \bar{s} - \underline{s} < \infty$ the gap between attack and benign attention logits, $\alpha_{\mathcal{A}} \leq k e^\gamma / (k e^\gamma + N - k) = \mathcal{O}(N^{-1})$.*

Theorem 2.5 (Padding invariance of the hard maximum). *Let $X' = \text{pad}(X, P)$ append P benign tokens to X . Then*

- (i) (Monotonicity, deterministic.) $a_h(X') = \max\{a_h(X), \max_{j \in \text{pad}} v_h^\top y_j\} \geq a_h(X)$: padding can only add candidates, never dilute the incumbent.
- (ii) (Exact invariance, high probability.) *Write the incumbent's margin over the benign mean as $\Delta_h = a_h(X) - \mu_b > 0$. Under Assumption 2.2 the padded scores are $\mu_b + \varepsilon_j$ with ε_j sub-Gaussian of proxy τ^2 , so*

$$\Pr[a_h(X') = a_h(X)] \geq 1 - P \exp\left(-\frac{\Delta_h^2}{2\tau^2}\right). \quad (5)$$

Hence a_h is exactly unchanged with probability at least $1 - \delta$ for every padding length $P \leq \delta \exp(\Delta_h^2/2\tau^2)$.

Proof. (i) The maximum over a union of index sets is the maximum of the sub-maxima, and $\mathcal{A} \subseteq \{1, \dots, N+P\}$ gives monotonicity. (ii) By (i), $a_h(X') \neq a_h(X)$ requires some padded token to strictly exceed the incumbent. A sub-Gaussian tail bound gives $\Pr[\varepsilon_j \geq \Delta_h] \leq \exp(-\Delta_h^2/2\tau^2)$ for each j , and a union bound over the P padded positions yields (5). Solving $P \exp(-\Delta_h^2/2\tau^2) \leq \delta$ for P gives the stated range. $\square$

Part (ii) is what makes the theorem non-vacuous: the tolerable padding budget grows *exponentially* in the squared signal-to-noise ratio Δ_h^2/τ^2 , so at any fixed margin the invariance survives context growth of many orders of magnitude. Read in the other direction, holding P fixed and asking what margin is required gives $\Delta_h \gtrsim \tau\sqrt{2\log(P/\delta)}$, the same $\sqrt{\log N}$ scale that reappears as a cost in Remark 2.6. The two statements are one phenomenon seen from opposite sides: the benign maximum creeps upward like $\tau\sqrt{2\log N}$, and that creep is precisely what eventually overtakes a fixed incumbent.

Remark 2.6 (The price: benign extremes). Theorem 2.5 is one-sided. The benign maximum also grows: $\mathbb{E}[\max_{j \leq N} \varepsilon_j] = \Theta(\tau\sqrt{2\log N})$, so a MultiMax threshold drifts as $t(N) = t_0 + \tau\sqrt{2\log N}$. This is a $\sqrt{\log N}$ correction where Prop. 2.3 demands $\sqrt{N}$; the hard maximum converts a polynomial degradation into a logarithmic one rather than removing length dependence.

3 Subgradient Sparsity and Normalised Boltzmann Annealing

Theorem 2.5 concerns the forward map only. The training dynamics are a separate matter, and they turn out to be where the hard maximum is weakest.

Theorem 3.1 (Subgradient support of the hard maximum). *Let $s_j = v_h^\top y_j$ and suppose the maximiser $j_h^* = \arg \max_j s_j$ is unique for each head h (which holds almost surely for continuously distributed features). Then the logit $z = \sum_{h=1}^H a_h + b$ of (2) satisfies*

$$\frac{\partial a_h}{\partial y_j} = v_h \mathbb{K}[j = j_h^*], \quad \text{hence} \quad |\text{supp}(\partial z / \partial y)| \leq H, \quad (6)$$

independently of N . By contrast, for softmax pooling with weights $\alpha_j = \text{softmax}_j(q^\top y_j / \sqrt{m})$ and pooled vector $\bar{y} = \sum_{j'} \alpha_{j'} y_{j'}$, the gradient is dense: for every j ,

$$\frac{\partial}{\partial y_j}(w^\top \bar{y}) = \alpha_j \left[w + \frac{1}{\sqrt{m}} (w^\top (y_j - \bar{y})) q \right] \neq 0 \quad \text{so} \quad |\text{supp}(\partial z / \partial y)| = N, \quad (7)$$

for all (w, q) outside a Lebesgue-null set.

Proof. For the maximum, write $a_h = \max_j s_j = s_{j_h^*}$ in a neighbourhood of the current iterate; uniqueness of j_h^* makes this identity locally exact, so a_h is differentiable there with $\partial a_h / \partial s_j = \mathbb{K}[j = j_h^*]$, and the chain rule through $s_j = v_h^\top y_j$ gives (6). Summing over h , the support of $\partial z / \partial y$ is contained in $\{j_1^*, \dots, j_H^*\}$, a set of at most H indices.

For softmax, differentiating $\alpha_{j'} = \text{softmax}_{j'}(q^\top y_{j'} / \sqrt{m})$ gives the standard Jacobian $\partial \alpha_{j'} / \partial y_j = \alpha_j (\delta_{jj'} - \alpha_{j'}) q / \sqrt{m}$. Hence

$$\begin{aligned} \frac{\partial}{\partial y_j}(w^\top \bar{y}) &= \alpha_j w + \frac{q}{\sqrt{m}} \sum_{j'} (w^\top y_{j'}) \alpha_j (\delta_{jj'} - \alpha_{j'}) \\ &= \alpha_j w + \frac{\alpha_j q}{\sqrt{m}} \left(w^\top y_j - \sum_{j'} \alpha_{j'} w^\top y_{j'} \right) = \alpha_j \left[w + \frac{1}{\sqrt{m}} (w^\top (y_j - \bar{y})) q \right]. \end{aligned} \quad (8)$$

The prefactor α_j is strictly positive for every j because the exponential is, so the derivative vanishes only if the bracket does, which forces $w \parallel q$. That condition is confined to a Lebesgue-null subset of (w, q) and, even there, is satisfied only at the isolated y_j solving the resulting scalar equation. Generically the support is all N positions. $\square$

Remark 3.2 (The bottleneck, quantified). Theorem 3.1 says each head learns from *one* token per sequence. The effective sample size per gradient step is therefore H rather than N : at $H=8$, $N=2048$ the gradient reaches $H/N = 0.39\%$ of positions, which is exactly what we measure (0.39% at $\tau=0$ against 100% at $\tau=0.5$). The consequence is not merely slow convergence. Before the argmax has locked onto $\mathcal{A}$, the single token receiving signal is a benign one chosen essentially at random, so the update carries no information about the concept; at strength 0.10 the un-annealed probe scored 0.00 recall at its *own training length*, a pure optimisation failure with no dilution component and invisible to the forward-only analysis of Theorem 2.5.

The obstruction is a property of the training objective, not of the deployed map, so it can be annealed away. Replace the maximum by the *normalised* Boltzmann operator [2]

$$\text{smax}_\tau(s) = \tau \log \left(\frac{1}{N} \sum_{j=1}^N e^{s_j / \tau} \right), \quad \text{smax}_\tau \xrightarrow{\tau \rightarrow 0} \max_j s_j, \quad \text{smax}_\tau \xrightarrow{\tau \rightarrow \infty} \frac{1}{N} \sum_j s_j, \quad (9)$$

The relaxation is useful because its gradient is dense. Differentiating (9),

$$\frac{\partial \text{smax}_\tau(s)}{\partial s_k} = \frac{\frac{1}{N} e^{s_k / \tau} \cdot \frac{1}{\tau}}{\frac{1}{N} \sum_{j=1}^N e^{s_j / \tau}} \cdot \tau = \frac{e^{s_k / \tau}}{\sum_{j=1}^N e^{s_j / \tau}} = \text{softmax}(s / \tau)_k > 0 \quad \text{for all } k, \quad (10)$$

so the support is all N positions, and $\sum_k \partial \text{smax}_\tau / \partial s_k = 1$: the operator distributes exactly one unit of gradient mass, reallocating it from the single argmax towards the whole sequence as τ grows. The $1/N$ factor

cancels in (10), which is why normalisation costs nothing in the backward pass while mattering a great deal in the forward one. Annealing $\tau \rightarrow 0$ concentrates $\text{softmax}(s/\tau)$ onto j^* , recovering both the hard forward map of (2) and the sparse subgradient of Theorem 3.1, so Theorem 2.5 still describes the shipped model exactly. *Remark 3.3* (Why the $1/N$ is load-bearing: length-invariance of the logit). Consider the null case in which head scores are i.i.d. with mean μ . Unnormalised LogSumExp gives

$$\text{LSE}_\tau(s) = \tau \log \sum_{j=1}^N e^{s_j/\tau} = \tau \log \left(N \cdot \frac{1}{N} \sum_j e^{s_j/\tau} \right) = \underbrace{\tau \log N}_{\text{grows with } N} + \text{smax}_\tau(s), \quad (11)$$

and summing over H heads the logit acquires an additive term $H\tau \log N$. Two consequences follow, and both are fatal for a probe that trains short and deploys long. First, the shift moves during training as τ anneals, so the bias b must be relearned at every epoch. Second, and worse, it moves with N : a probe fitted at N_{train} meets an offset $H\tau \log(N_{\text{deploy}}/N_{\text{train}})$ larger at deployment, which no amount of in-distribution calibration anticipates. Taking expectations in (9) instead, $\mathbb{E}[\text{smax}_\tau(s)]$ depends on N only through the extreme-value term of Remark 2.6, which is $\Theta(\sqrt{\log N})$ rather than $\Theta(\log N)$ and carries no τ or H multiplier.

The prediction is quantitative and we confirmed it: $H\tau \log N$ at $H=8$, $\tau=1$, $N=512$ evaluates to 49.9 against a measured +42, and at $N=16,384$ to 77.6 against a measured +68. The observable symptom before the fix was a recall curve non-monotone in signal strength (1.00 at $S=0.15$ but 0.04 at $S=0.50$), a bug signature rather than an effect, since no dilution mechanism improves with a weaker needle.

4 Methodology

Sections 2–3 fix the forward map and its training dynamics. This section states the remaining choices explicitly, so that every constant used in Sections 5 and 6 is either derived here or selected by the ablation of §5.6 rather than asserted.

4.1 Token transform, head projection, and hyperparameters

The token transform of (1) is a single hidden layer,

$$\phi(x_j) = \text{GELU}(W_1 x_j + b_1) \in \mathbb{R}^m, \quad W_1 \in \mathbb{R}^{m \times d}, \quad (12)$$

followed by the head projection $s_{j,h} = v_h^\top \phi(x_j)$ with $V = [v_1, \dots, v_H] \in \mathbb{R}^{m \times H}$ carrying no bias, since a per-head bias is absorbed by the scalar b in (2). Every aggregator in this paper shares this ϕ and differs only in the reduction, which is what makes the comparison a comparison of reductions.

We use $m = 512$, $H = 8$, $c = 10$ and $C = 4096$ throughout. These are not free: c and H are selected by the grid of §5.6, which shows a usable window rather than a monotone preference, and C trades memory against occupancy along the measured law $\Theta(\min(C, N))$ of Remark 5.1. Compute runs in bf16 with the reduction accumulated in fp32; §5.7 shows the choice of compute precision does not change the gradient conclusion.

4.2 Annealing schedule

Training replaces the hard maximum by the normalised Boltzmann operator of (9) under a fixed schedule. Writing t for the epoch index and T for the total number of epochs,

$$\tau(t) = \begin{cases} \tau_0 \left(1 - \frac{t}{\rho T}\right), & t < \rho T, \\ 0, & t \geq \rho T, \end{cases} \quad \tau_0 = 1, \quad \rho = 0.6. \quad (13)$$

The schedule is linear rather than exponential and it reaches exactly zero before training ends. Both details matter. A schedule that only approaches zero leaves a train-to-deploy mismatch, because deployment uses the exact maximum; annealing to zero with 40% of the epochs remaining lets the probe adapt to the operator it will actually be evaluated under. The deployed model is therefore $\tau = 0$ in every number we report.

Streaming MultiMax: activation overhead $\Theta(\min(C, N))$, independent of N once $C < N$

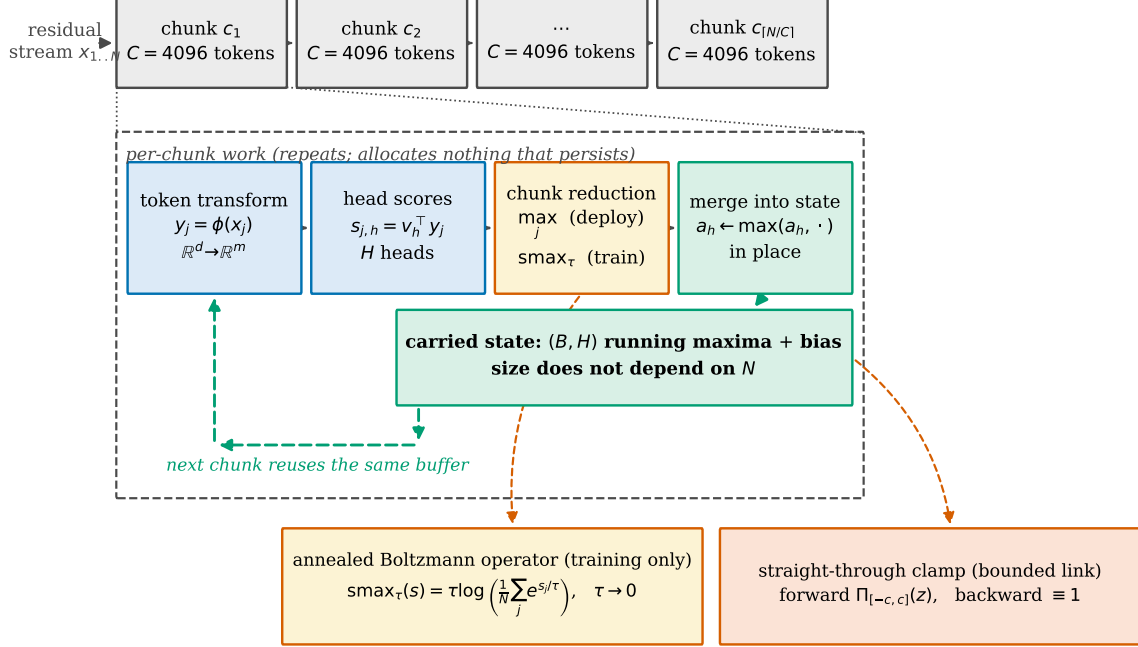


Figure 1: The streaming MultiMax pipeline. Chunks of C tokens enter one at a time; the token transform ϕ and the head projection are applied within a chunk and discarded, and the only quantity that crosses a chunk boundary is the (B, H) buffer of running maxima. Its size is set by H , not by N , which is the whole of the memory claim. The two boxes at the foot are training-time mechanisms that leave the deployed map unchanged: the $1/N$ inside the Boltzmann operator of (9) removes an $H\tau \log N$ length-dependent offset that plain LogSumExp would introduce (Remark 3.3), and the straight-through clamp keeps gradients alive in the saturated regime where a plain clamp returns exactly zero (Remark 2.1).

4.3 Calibration and the cascading gate

The gate escalates iff $|\hat{p} - \frac{1}{2}| < \delta$. This is a statement about $\hat{p}$, so it is meaningful only if $\hat{p}$ is calibrated [12]. *Remark 4.1* (Temperature scaling is structurally insufficient here). A MultiMax logit is a sum of H *maxima*, and by Remark 2.6 each carries a positive, N -dependent offset even under the null. Temperature scaling $\hat{p} = \varsigma(z/T)$ is a pure rescaling; it fixes $\varsigma^{-1}(\hat{p}) = 0$ at $z = 0$ and cannot move a mis-centred boundary. We observed the pathology directly: NLL 0.436 (informative) alongside accuracy 0.500 at threshold 0 (unusable). Two-parameter Platt scaling [22], $\hat{p} = \varsigma(az + b)$, absorbs both scale and offset and is the minimal correction. Fitted by L-BFGS on held-out logits, Platt scaling reduces Brier score [5] from 0.0538 to 0.0122, NLL from 0.1705 to 0.0474, and accuracy from 0.931 to 0.988.

Budget-driven thresholding. A fixed δ is the wrong interface. On a well-calibrated accurate probe the genuinely ambiguous region is narrow, and $\delta=0.05$ escalated 0 of 160 held-out examples. That is correct behaviour, but it means a hard-coded δ silently degenerates to probe-only on a good probe and to LLM-only on a bad one. Deployments have an escalation *budget*, so we select δ as the target-rate quantile of $|\hat{p} - \frac{1}{2}|$. Targets of 1/2/5/10% land at 1.2/2.5/5.0/10.0%. Total system cost is

$$C_{\text{total}}(\delta) = c_p + u(\delta) c_L, \quad u(\delta) = \Pr[|\hat{p} - \frac{1}{2}| < \delta], \quad (14)$$

so with $c_p \ll c_L$ the saving is governed almost entirely by the escalation rate.

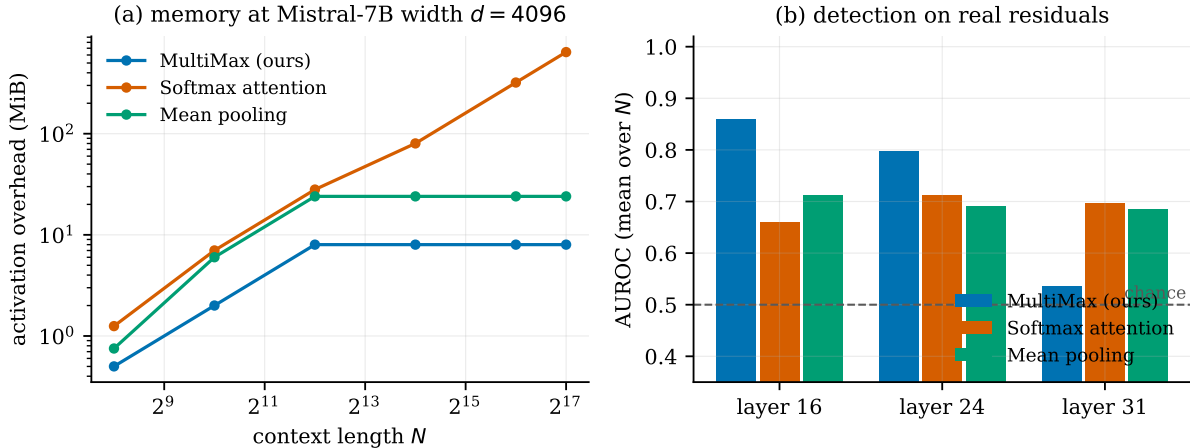


Figure 2: Mistral-7B-v0.1 residual streams. (a) Activation overhead at the model’s true width $d=4096$: MultiMax is flat at 8.0 MiB from $N=4096$ to 131,072, a $32\times$ increase in length at zero increase in footprint, while softmax pooling rises from 28.0 to 641.0 MiB. (b) Detection AUROC by layer, averaged over $N \in \{256, 1024, 4096\}$. The hard maximum is the better reduction at layers 16 and 24 and loses its advantage at layer 31.

5 Real-Model and Systems Experiments

Setup. Two families of experiment. §5.1 reads *real* residual streams out of an open-weight model. The remainder use synthetic residual streams, $d=2048$, batch 1, bf16 compute with fp32 reduction, chunk $C=4096$, on a single 7.96 GiB consumer GPU; the attack is a fixed direction scaled to unit magnitude per coordinate against background $\sigma=0.5$. Probes train only at $N=512$, so long context is strictly out of distribution. Recall is read at a threshold giving 1% FPR on benign sequences *of the same length*, so no length-dependent score shift contaminates the comparison.

5.1 Real open-weight residual streams

Protocol. We extract residual streams from Mistral-7B-v0.1 at layers 16, 24 and 31 of 32, for sequences of $N \in \{256, 1024, 4096\}$ tokens. Positives embed a short harmful-intent request at a uniformly random position in benign filler; negatives embed a topically matched benign request of similar length in the same filler, so intent rather than length or topic is the discriminating variable. Length matching is not a detail: an earlier synthetic protocol that did not match it scored AUROC 0.685 on needle length alone. Probes are trained on half the sequences and scored on the held-out half.

Two constraints we did not choose. Llama-3-8B-Instruct was the intended model and every `meta-llama` repository returns HTTP 403 for our account, so we use Mistral-7B-v0.1, which is ungated and has exactly 32 layers. More importantly, Mistral-7B-v0.1 has a 32,768-token position range, so $N = 2^{17}$ is not a length at which real residuals for this model *exist*. Real-residual detection therefore stops at $N = 4096$ and the ladder to 131,072 in Fig. 2(a) and §5.3 uses tensors at the model’s true width. That split is deliberate: a probe’s memory profile is a function of (d, N, C) and the reduction, and does not depend on whether the numbers came from a forward pass, whereas detection obviously does. We do not report detection at lengths where we cannot measure it.

A leak we found and closed. Our first split held out sequences by index. Since each of the 12 needle sentences recurs four times across 48 sequences, both halves contained every needle, and the probe was scored on sentences it had trained on, differing only in filler placement. That measures memorisation of 12 strings, not generalisation to unseen harmful intent, and it inflated MultiMax by +0.106 mean AUROC (0.837 against the 0.731 in Table 1). Softmax pooling gained +0.036 and mean pooling actually lost 0.017, so the leak flattered the hard maximum specifically. Every real-model number we report is from the needle-disjoint split; the index-split figures are retained in the artifact for comparison. We flag this because the size of the effect,

N	layer	MultiMax (ours)	Softmax attention	Mean pooling
256	16	0.917	0.823	0.828
256	24	0.844	0.856	0.774
256	31	0.544	0.812	0.722
1024	16	0.906	0.457	0.500
1024	24	0.783	0.642	0.655
1024	31	0.476	0.703	0.759
4096	16	0.757	0.701	0.807
4096	24	0.764	0.639	0.646
4096	31	0.586	0.576	0.575
mean over all cells		0.731	0.690	0.696

Table 1: Detection AUROC on real Mistral-7B-v0.1 residual streams under a *needle-disjoint* split: the six harmful sentences used at test time never appear in training. 24 positives and 24 negatives per cell. MultiMax wins 5 of 9 cells and the mean, by a narrow margin.

a tenth of an AUROC point, is larger than most of the differences such tables are used to argue about.

Result. On the honest split MultiMax still leads, with mean AUROC 0.731 against 0.690 for softmax pooling and 0.696 for mean pooling, and wins 5 of the 9 cells. That is a real but modest lead, and considerably narrower than the synthetic benchmarks suggest. The systems claim survives far more cleanly than the detection claim: at the model’s true width the MultiMax overhead is flat at 8.0 MiB across a $32\times$ range of N , while softmax pooling grows from 28.0 MiB to 641.0 MiB, a ratio of $80\times$ at $N = 131,072$. The asymmetry is the paper’s thesis in miniature. The architectural claim transfers to real activations essentially unchanged; the detection advantage transfers, but shrinks.

Where it fails, and what that tells us. MultiMax degrades sharply with depth. Averaged over lengths it scores 0.860 at layer 16, 0.797 at layer 24 and 0.535 at layer 31, which is indistinguishable from chance; at $N=1024$, layer 31 it reaches 0.476, slightly *worse* than chance, while mean pooling on the same activations reaches 0.759. We read this as the final-layer residual stream being dominated by next-token prediction structure, in which the single most extreme coordinate along a learned direction is a poor summary of a diffuse property like intent. This is a concrete, model-side instance of the limit the paper argues for throughout: the hard maximum is the right reduction when evidence is concentrated and the wrong one when it is spread. It also gives a deployment rule that costs nothing to follow, namely to site the probe mid-stack rather than at the final layer.

5.2 Batched systems scaling

Every systems number above and in §5.3 is at batch size 1, and deployments batch. Table 2 sweeps $B \in \{1, 2, 4, 8, 16\}$ at $N=65,536$, reporting overhead above the input tensor rather than total allocation. The distinction matters: the input is a cost the serving stack already pays, and including it compresses every aggregator toward the same number and hides the difference between them.

B	MultiMax (ours)		Softmax attention		Mean pooling	
	MiB/seq	latency	MiB/seq	latency	MiB/seq	latency
1	8.0	4.24 ms	320.5	6.96 ms	24.0	4.47 ms
2	20.0	12.35 ms	320.5	13.89 ms	22.0	12.88 ms
4	20.0	24.14 ms	320.5	27.95 ms	20.0	27.28 ms
8	20.0	50.54 ms	320.5	50.02 ms	20.0	53.63 ms
16	20.0	92.68 ms	320.5	320.57 ms	20.0	105.31 ms

Table 2: Batched cost at $N=65,536$, overhead above the input tensor, per sequence. MultiMax and mean pooling settle at 20.0 MiB per sequence from $B=2$ onward; softmax pooling holds 320.5 MiB per sequence at every batch size. At $B=16$ the totals are 320 MiB against 5128 MiB, a factor of 16.

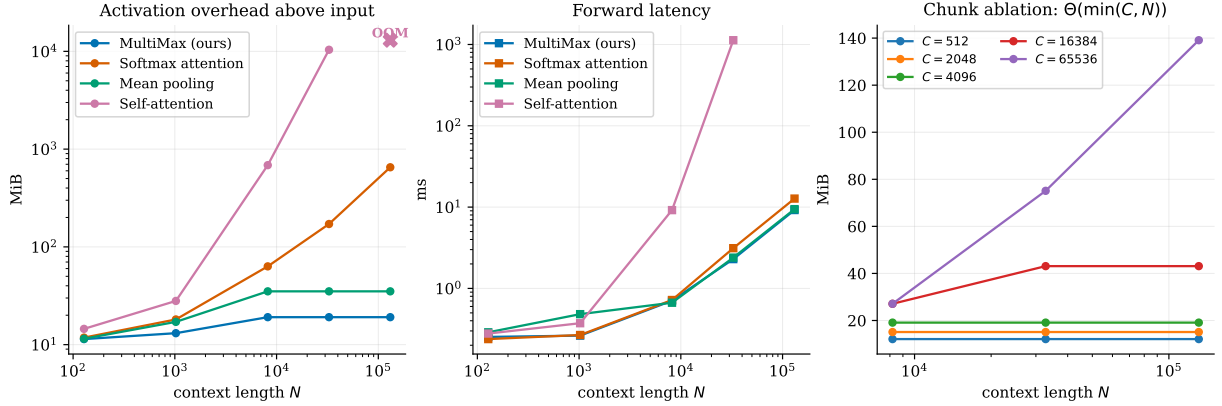


Figure 3: Left, centre: activation overhead above the input tensor, and forward latency, versus N . Right: the chunk ablation, in which rows are flat in N once chunking engages and columns rise with C .

Probe	$N=8,192$	$N=32,768$	$N=131,072$	overhead @131k	α
MultiMax (ours)	0.707 ms	2.293	9.205	19.1 MiB	0.000
Mean pooling	0.667	2.392	9.438	35.1 MiB	0.000
Softmax attention	0.720	3.125	12.729	652.1 MiB	0.842
Self-attention	9.182	1128.474	OOM	10381.6 MiB	1.960

Table 3: Latency and activation overhead. α is the log-log exponent of overhead in N , fitted on $N \geq 8192$. Overhead excludes the input tensor, which is unavoidably $\Theta(N)$ and would otherwise dominate.

Two things are worth separating here. The per-sequence advantage over softmax pooling is $16\times$ and holds at every batch size, which is the result the memory law predicts. The per-sequence figure for MultiMax is *not* constant in B , though: it rises from 8.0 MiB at $B=1$ to 20.0 MiB at $B=2$ and then stays there. We attribute the step to allocator behaviour rather than to the reduction, since the carried state is (B, H) and scales exactly linearly by construction, but we report the measurement rather than the model. The latency separation appears only at the top of the sweep: MultiMax and softmax pooling are within 1% of each other at $B=8$ and differ by $3.5\times$ at $B=16$ (92.68 against 320.57 ms), which is where the softmax intermediate stops fitting comfortably and the kernel starts paying for it.

5.3 Benchmark A: systems scaling

MultiMax overhead is flat at 19.1 MiB across a $16\times$ length increase (Table 3). Self-attention is $492\times$ slower than MultiMax at $N=32,768$ and then exhausts memory. We note a correction to a common framing: *single-query* attention pooling is $\Theta(N)$, not $\Theta(N^2)$; its score tensor is $(1, N)$. Only self-attention is quadratic. *Remark 5.1* (Is $\mathcal{O}(1)$ real, or an artifact of chunking?). A reviewer should press on this, and the honest answer is that the suspicion is half right. The flat footprint *is* a consequence of streaming (that is the mechanism, not an artifact), and it obeys a measurable law. Sweeping $C \times N$ (Fig. 3, right) gives row spread exactly 0.0 MiB for every $C < N$ ($\alpha = 0.000$ at $C \in \{512, 2048, 4096\}$ across N from 8,192 to 131,072), while overhead rises with C ($\beta = 0.510$). For $C \geq N$ there is one chunk and the chunk *is* the sequence, so overhead tracks N by construction. The law is $\Theta(\min(C, N))$. The memory is not free; it is $\Theta(C)$ memory the operator chooses, and $\Theta(1)$ in N , the one quantity the adversary controls.

5.4 Benchmark B: weak-signal detection

With annealing, MultiMax attains full recall at every strength and leads softmax pooling in the weak-signal regime (1.00 vs 0.68 at $S=0.10$). Mean pooling never recovers, as Prop. 2.3 requires. Without annealing the same architecture scores 0.00/0.00 at $S=0.10$: the gap between the two MultiMax rows in Fig. 4 is the entire contribution of §3.

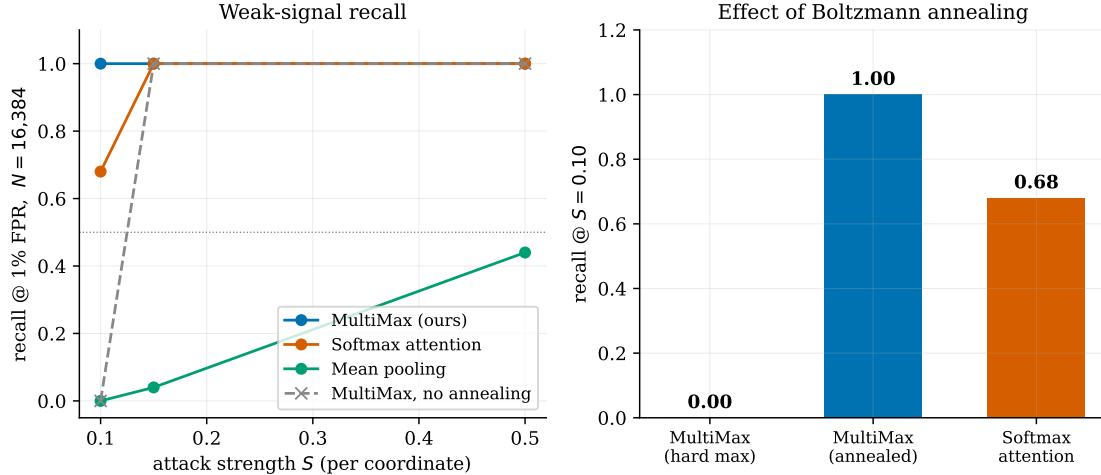


Figure 4: Recall at 1% FPR, $N=16,384$, trained at $N=512$. Annealing converts a total training failure at $S=0.10$ into full recall.

Strength S	MultiMax (ours)	Softmax attention	Mean pooling
0.10	1.00 / 1.00	0.68 / 1.00	0.00 / 0.52
0.15	1.00 / 1.00	1.00 / 1.00	0.04 / 0.94
0.50	1.00 / 1.00	1.00 / 1.00	0.44 / 1.00

Table 4: Recall@16,384 / recall@512. The second entry separates “never learned the concept” from “learned it, then diluted”. Dilution ratio $k/N = 2.4 \times 10^{-4}$.

5.5 Normalisation, drift, and what calibration transfer does not show

Three analytic claims in §2–§3 had never been checked against their own predictions. Two are confirmed and the third is not established, which we report as such.

The normalisation term is exact. Evaluating both operators on identical head scores, the measured offset between unnormalised and normalised LogSumExp matches $H\tau \log N$ to within 3.6×10^{-6} at every (N, τ) tested, across $N \in [512, 131,072]$ and $\tau \in \{0.5, 1\}$. At $N=131,072$, $\tau=1$ the offset is 94.27 logits. This is not an approximation that happens to be close: it is an identity, and it is why Remark 3.3 treats the $1/N$ as load-bearing rather than cosmetic.

The drift is the predicted shape, with a slope 7% above theory. Fitting $\mathbb{E}[\max_j \varepsilon_j] = a + b\sqrt{\log N}$ over $N \in [10^2, 10^5]$ gives $b = 1.513$ against the asymptotic $\tau\sqrt{2} = 1.414$, with $R^2 = 0.999$. The functional form of Remark 2.6 is confirmed; the slope sits slightly high and the fitted intercept is negative (-0.735), which is expected, since the asymptotic expression drops lower-order terms that are still material at $N = 10^2$.

A claim we cannot support. §4.3 recalibrates per length. We tested whether that is *necessary* by fitting one Platt map at $N=4096$ and applying it unchanged at 1k, 32k and 128k. The mean logit does drift exactly as predicted, rising by 1.52 from $N=1024$ to $N=131,072$. But the transferred map loses nothing measurable: Brier stays at 0.0000 and accuracy at 1.000 at every length, and the gap to per-length recalibration is 1.6×10^{-8} . The reason is uninteresting and worth stating plainly, namely that at these signal strengths the classes are separated well enough that a shift of 1.5 logits never moves the boundary between them. So the drift is real and measured, and the case for per-length recalibration is *not* established by this experiment. We keep the per-length protocol because it is free and because Remark 2.6 predicts the drift will eventually bite at weaker separation, but we no longer present it as an empirically forced choice.

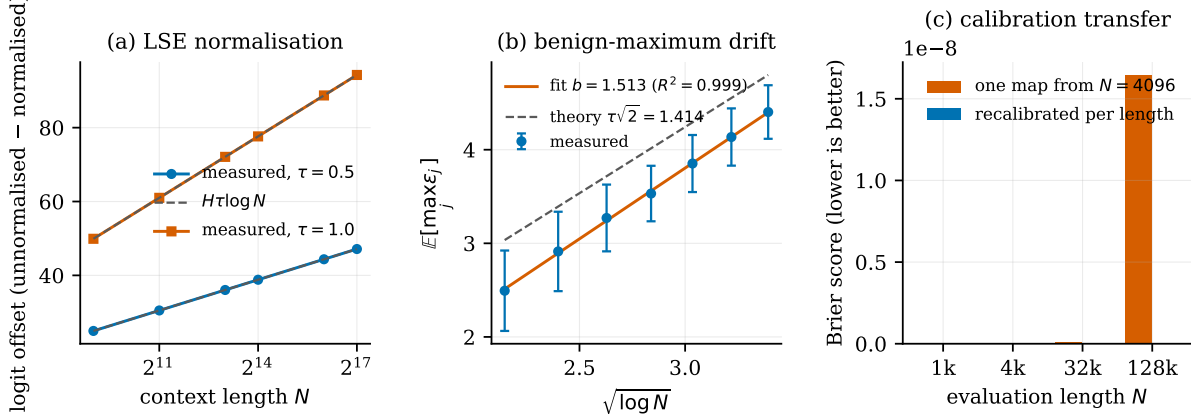


Figure 5: (a) The offset between unnormalised and normalised LogSumExp, against the closed form $H\tau \log N$. (b) The benign maximum against $\sqrt{\log N}$, with the least-squares fit and the asymptotic prediction. (c) Brier score at each evaluation length under a single Platt map fitted at $N=4096$ and under per-length recalibration.

clamp c	head count H				
	1	4	8	16	32
1	0.500	0.500	0.500	0.500	0.500
2	0.500	0.500	0.500	0.500	0.500
5	1.000	0.975	0.500	0.500	0.500
10	1.000	1.000	1.000	1.000	1.000

Table 5: Out-of-distribution AUROC, trained at $N=512$ and evaluated at $N=16,384$ at weak signal $S=0.15$. Bold marks cells above chance. The boundary is a staircase in (c, H) , not a preference for either parameter alone: every H reaches 1.000 at some c , and no c below 5 works at any H .

5.6 Hyperparameter ablation: the bounding link and the head count

The grid is not the smooth trade-off one might expect. Every cell at $c \leq 2$ scores *exactly* 0.500, and so do $H \geq 8$ at $c = 5$. Exactly chance, with the identical training loss across all H at fixed c (0.8133 at $c=1$, 1.1269 at $c=2$), is the signature of a constant output rather than of a weak one.

The mechanism, measured rather than assumed. The suspect is the bounded link. The logit of (2) is a *sum* of H per-head maxima, so $|z|$ scales with H ; if it exceeds c for every input, the clamp maps every sequence to the same value, all scores tie, and AUROC is 0.500 by construction. We tested this by recording the *pre-clamp* logit at both lengths rather than inferring it (Table 6).

The measurement confirms the mechanism and corrects our first guess about it. We expected saturation to be a deployment-length effect, appearing at $N=16,384$ in probes that were healthy at $N=512$. It is not. Every failing cell is already pinned at the training length, with a mean pre-clamp $|z|$ of 1605.6 at $c=1$, 344.3 at $c=2$ and 69.9 at $c=5$, in each case one to two orders of magnitude outside the interval. These probes never escape the flat region at all; the identical training losses across H at fixed c are the constant-output signature of that, and the low absolute loss at $c=5$ (0.0216) simply reflects a constant that happens to sit near the class balance.

Saturation alone is not fatal. The last row of Table 6 is the reason to measure rather than argue. At $c=10$, $H=32$ the positives are fully saturated at $N=16,384$ ($|z| = 11.4 > c$) and the probe still scores AUROC 1.000. The naive reading, saturation implies chance, is therefore false. What destroys ranking is not saturation of one class but saturation of *both* onto the same bound: while the negatives remain inside the interval, the clamp still separates the classes. This also explains why H enters the staircase only through $|z|$: raising H raises the logit scale, which brings the positive class to the bound first and both classes to the bound soon

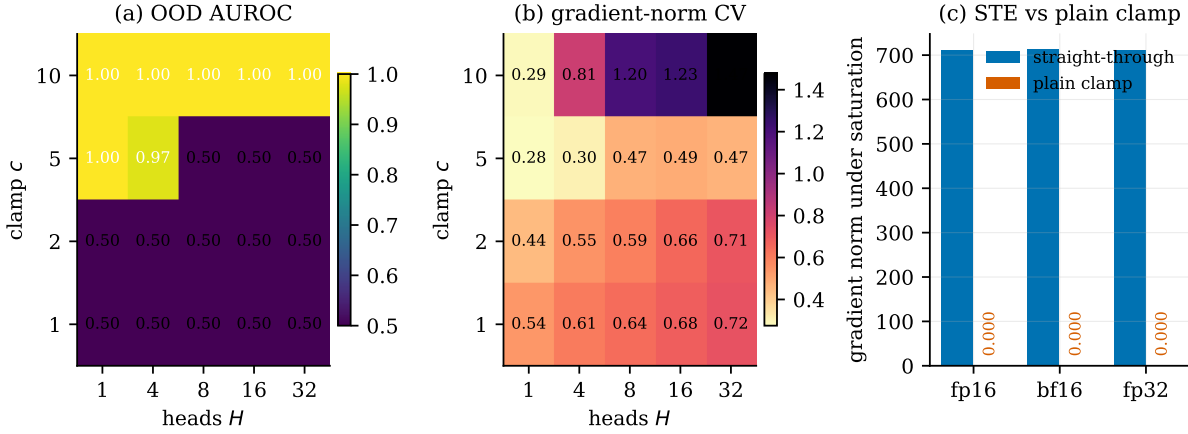


Figure 6: (a) Out-of-distribution AUROC over the $c \times H$ grid: probes train at $N=512$ and are scored at $N=16,384$. The usable region is a staircase, and the c required grows with H . (b) Coefficient of variation of the gradient norm across training, a scale-free stability measure comparable across H . (c) Gradient norm under deliberate saturation, by compute precision.

c	H	AUROC	at $N=512$ (train)		at $N=16,384$	
			mean $ z $	saturated	mean $ z $	saturated
1	8	0.500	1605.6	1.00	1605.9	1.00
2	8	0.500	344.3	1.00	348.1	1.00
5	8	0.500	69.9	1.00	70.6	1.00
10	1	1.000	3.5	0.00	3.5	0.00
10	8	1.000	6.9	0.00	7.5	0.00
10	32	1.000	7.8	0.00	11.4	1.00

Table 6: Pre-clamp logit magnitude and saturated fraction, measured on positive sequences. Every chance-level cell is pinned far outside $[-c, c]$ at *both* lengths, so the failure is a training pathology rather than a length-generalisation one. The last row is the informative exception and is discussed in the text.

after.

What this means for practice. c is not a free safety margin to be set small. It must be large enough that the two classes are not both pinned at the operating length, and since $|z|$ grows with H the required c grows with H too. Our default $c=10$, $H=8$ sits inside the usable region with margin, and $c=10$ succeeded at every H we tried. The practical check is cheap and does not require labels: monitor the fraction of pre-clamp logits outside $[-c, c]$ and the spread of z within each class, rather than waiting for AUROC to collapse.

5.7 Compute precision and the straight-through clamp

Remark 2.1 claims a plain clamp zeroes the gradient in the saturated region and the straight-through estimator does not. That was measured once, in one precision. Table 7 repeats it in fp16, bf16 and fp32 under deliberate saturation, a raw logit of about 46.5 against $c=10$.

The outcome is uniform: the plain clamp yields exactly 0.000 gradient norm and a non-zero gradient fraction of 0.000 in all three precisions, while the straight-through version keeps the fraction at 1.000 with a gradient norm near 712 in each. No output was non-finite in any configuration. This is a negative result about precision and a positive one about the estimator: the failure Remark 2.1 identifies is structural, arising from the derivative $\mathcal{K}[|z| < c]$ rather than from rounding, so it cannot be escaped by changing dtype and does not need to be.

compute dtype	raw logit	gradient norm		non-zero gradient fraction	
		straight-through	plain clamp	straight-through	plain clamp
fp16	46.54	7.119×10^2	0.000	1.000	0.000
bf16	46.59	7.121×10^2	0.000	1.000	0.000
fp32	46.54	7.116×10^2	0.000	1.000	0.000

Table 7: Gradient survival under saturation, by compute precision. All gradients were finite in every configuration. The result is precision-independent: the plain clamp is dead in all three, and the straight-through estimator keeps every parameter receiving gradient in all three.

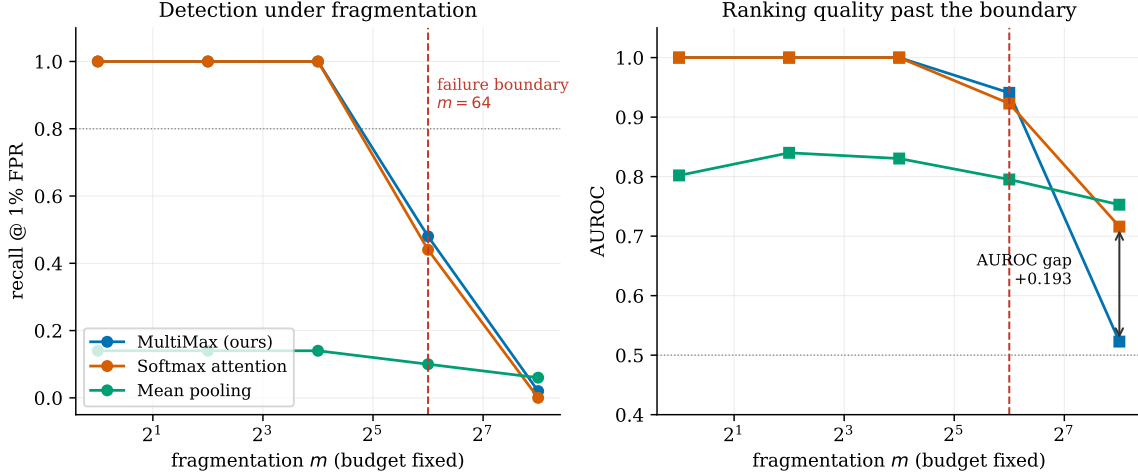


Figure 7: Fragmented attack: a fixed budget $S=2.0$ split across m non-contiguous tokens. Both pooled aggregators fail at $m=64$; past the boundary softmax retains markedly better ranking.

6 Adversarial Fragmentation and Cascades

The maximum has one structural adversary: an attacker who keeps the total injected budget fixed and spreads it thinly enough that no single token stands out. This section measures that attack under progressively fairer conditions, tests the two countermeasures the paper previously only proposed, and replaces the simulated second stage of the cascade with a real guardrail model.

6.1 Benchmark C: distributed attacks with known m

An adversary who holds the total budget fixed but spreads it over m non-contiguous tokens presents per-token magnitude S/m , so

$$\max_j v_h^\top y_j = \Theta(S/m), \quad \text{while} \quad \frac{1}{N} \sum_j v_h^\top y_j = \Theta(S/N) \text{ is unchanged in } m, \quad (15)$$

the exact dual of Prop. 2.3. This is the attack aimed at a maximum, and it is the structural price of the hard reduction.

Three points, all against our own interest. First, the predicted MultiMax-specific vulnerability *did not isolate*: both peaked aggregators fail at the same m , so at this budget the distributed attack degrades query-based pooling as much as the hard reduction. Second, the recall boundary hides a real difference. At $m=256$ both sit at recall ≈ 0 under a 1% FPR threshold, but AUROC is 0.523 for MultiMax against 0.716 for softmax: softmax keeps usable ordering after the thresholded detector has failed, while the hard maximum loses its signal more completely.

Third, and this is the result we least expected, mean pooling is *invariant* to fragmentation, exactly as

m	per-token S/m	MultiMax (ours)		Softmax attention		Mean pooling	
		recall	AUROC	recall	AUROC	recall	AUROC
1	2.0000	1.000	1.000	1.000	1.000	0.140	0.802
16	0.1250	1.000	1.000	1.000	1.000	0.140	0.830
64	0.0312	0.480	0.941	0.440	0.923	0.100	0.795
256	0.0078	0.020	0.523	0.000	0.716	0.060	0.753

Table 8: Recall at 1% FPR and AUROC under fragmentation, $N=16,384$, fixed budget $S=2.0$. Failure boundary (recall < 0.80): MultiMax $m=64$, softmax $m=64$, mean pooling $m=1$. Bold marks the boundary row and the best AUROC at $m=256$.

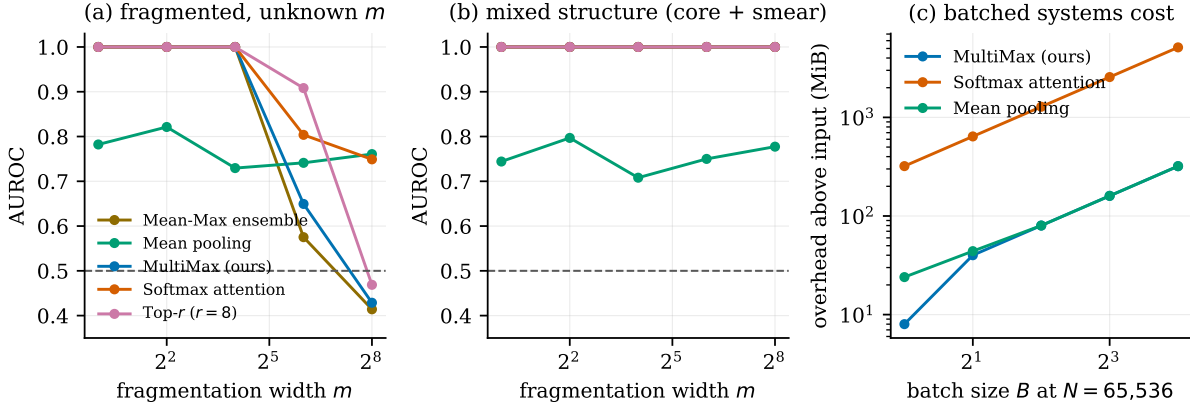


Figure 8: (a) AUROC against fragmentation width under an unknown- m protocol: probes train on a mixture $m \in \{1, 8, 64\}$ and are evaluated at held-out widths. (b) The mixed adversary, which retains a concentrated core carrying half the budget. (c) Per-sequence overhead against batch size at $N=65,536$.

(15) predicts. Its AUROC moves only within 0.753–0.840 across a $256\times$ spread of the same budget, while MultiMax falls from 1.000 to 0.523 over the same range. The aggregator that is strictly worst at $m=1$ (AUROC 0.802 against 1.000) is strictly best at $m=256$ (0.753 against 0.716 and 0.523), and the crossing is not a tie-break but a consequence of the structure: a mean reads accumulated deviation, which is the one functional a budget-preserving adversary cannot reduce. This is the sharpest evidence in the paper that our recommendation must be architectural rather than a claim of dominance; it also converts the ensembling suggestion of §7.7.2 from speculation into a measured complementarity.

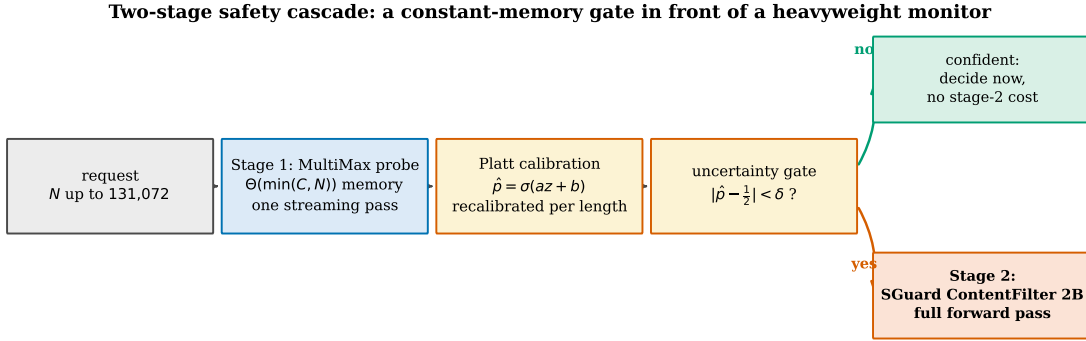
6.2 Unknown m , mixed structure, and the two proposed countermeasures

Benchmark C trains on the same m it evaluates, which hands the defender knowledge the adversary chooses. Here probes train on a mixture $m \in \{1, 8, 64\}$ and are scored at $m \in \{1, 4, 16, 64, 256\}$, so three of the five widths are held out. We add the two countermeasures §7.7.2 proposed but never tested: streaming Top- r with $r=8$, and a Mean-Max ensemble carrying separate head matrices for a running maximum and a running mean.

Top- r works, and it works past where it should. Replacing the maximum by the mean of the r largest scores lifts AUROC at $m=64$ from 0.649 to 0.908, recovering most of the gap to softmax pooling while keeping the streaming property, since a running top- r buffer is $\Theta(r)$ state. The naive prediction was that Top- r should help up to about $m \approx r = 8$ and no further; it in fact helps substantially at $m=64$, eight times that. We do not have a clean account of why, and flag it as the most promising loose thread here. At $m=256$ the advantage is gone (0.469 against 0.429), so Top- r moves the boundary rather than removing it.

probe	fragmentation width m (held out except $m=1, 64$)				
	1	4	16	64	256
MultiMax (ours)	1.000	1.000	1.000	0.649	0.429
Top- r ($r=8$)	1.000	1.000	1.000	0.908	0.469
Mean-Max ensemble	1.000	1.000	1.000	0.575	0.414
Softmax attention	1.000	1.000	1.000	0.804	0.749
Mean pooling	0.782	0.821	0.729	0.741	0.761

Table 9: AUROC under unknown- m fragmentation at $N=16,384$ with a fixed budget $S=2.0$. Bold marks the best probe at the two widths past the failure boundary. No aggregator is best everywhere, and the ordering inverts across the boundary.



δ is the target-rate quantile of $|\hat{p} - \frac{1}{2}|$, so the escalation budget is set by the operator, not by a hard-coded constant.
 Stage 2 runs on the escalated fraction only, so the compute saved is $(1 - \text{escalation rate})$ of the stage-2 cost.

Figure 9: The evaluated cascade. Stage 1 is a MultiMax probe over Mistral-7B residuals, Platt-calibrated and gated on $|\hat{p} - \frac{1}{2}| < \delta$; stage 2 is SGuard-ContentFilter-2B, run in full. Escalated traffic pays the stage-2 cost, confident traffic does not.

The Mean-Max ensemble does not work, and we predicted that it would. §7.7.2 argued that because the maximum and the mean fail on disjoint geometries, their sum should be at least as good as either member everywhere. It is not. At $m=64$ Mean-Max scores 0.575, *below* the plain maximum’s 0.649, and at $m=256$ it scores 0.414 against mean pooling’s 0.761. Concatenating the two reductions and training end to end lets the optimiser lean on whichever member is stronger on the training mixture, which here is the maximum, and the mean-pooled heads never receive the gradient signal that would make them useful at widths absent from training. Ensembling aggregators may still be right, but it evidently requires the members to be trained or weighted separately rather than summed inside one objective, and the claim in §7.7.2 should be read as refuted in the form we stated it.

A mixed adversary is easier, not harder. We also tested an adversary splitting the budget between a concentrated four-token core and a diffuse smear over $4m$ tokens. Every peaked aggregator scores 1.000 at every m against it (Fig. 8(b)). This is the expected result once stated: a maximum needs only one token above threshold, so an adversary who keeps any concentrated component concedes the detection. It also sharpens what the fragmentation attack requires. The attacker must spread the budget *uniformly*; a hybrid that hedges is strictly worse for them than committing, which narrows the space of effective attacks against a hard maximum considerably more than §6.1 suggested.

6.3 A concrete cascade with a deployed guardrail as stage 2

The cascade in §4.3 used a deterministic stand-in for stage 2 and said so. Every cost and safety number it produced was therefore a statement about the stand-in. Here stage 2 is SGuard-ContentFilter-2B [17], a

deployed guardrail model, run for real on the same 48 held-out sequences. SGuard emits one special token per risk category, so we read the softmax over each category’s {safe, unsafe} token pair directly from the logits and take the maximum across categories, giving a genuine probability rather than a parsed string.

operating point	escalated	AUROC	ASR	FLOPs vs. always-on	latency
probe only	0.000	0.554	0.500	0.0010	0.37 ms
cascade, 50% budget	0.500	0.625	0.208	0.5005	798 ms
SGuard on everything	1.000	0.997	0.333	1.0000	1595 ms

Table 10: End-to-end cascade against a real stage 2, $N=512$, 48 held-out sequences. ASR is the fraction of harmful sequences the composed system fails to flag. Escalation budgets below 50% are omitted because they escalate nothing; see the text.

The economics are as claimed. A stage-1 pass costs 0.104% of a stage-2 pass in FLOPs and 0.37 ms against 1595 ms in measured latency, a factor of about 4300. That gap is the entire reason to build a cascade, and it is larger than the paper’s cost model assumed.

The gate, at this stage-1 quality, does not work. Every escalation budget from 1% to 25% escalates *nothing*. The cause is visible in the fitted Platt parameters: $a = 95.9$ maps almost every logit to $\hat{p} \in \{0, 1\}$, so $|\hat{p} - \frac{1}{2}|$ is near $\frac{1}{2}$ everywhere and no quantile of it produces a usable band. A weak stage 1 is not merely a weak detector; it destroys the uncertainty signal the gate depends on. This is a failure mode §4.3 did not anticipate, and it is a direct consequence of stage 1’s AUROC being 0.554 here, far below the 0.917 it reaches at layer 16 in Table 1. We report the cascade at the configuration we actually measured rather than at the probe’s best layer.

An unexpected complementarity. At a 50% budget the composed system reaches an attack success rate of 0.208, *better* than running SGuard on all traffic (0.333) at half the compute. That is not a rounding artifact: SGuard itself misses a third of these attacks, because a single harmful sentence buried in benign filler is exactly the long-context dilution failure Chen et al. [7] document, and some of the sequences it misses are ones the probe catches. The two stages fail on partly disjoint inputs. We are wary of over-reading a result at $n=48$ with a weak stage 1, and state it as a hypothesis worth testing properly rather than as a headline: a cascade may be worth building for coverage as well as for cost, because a cheap monitor with different failure modes is not merely a cheaper version of the expensive one.

7 Related Work and Discussion

7.1 Mechanistic interpretability and linear probing

Reading a concept off intermediate representations with a trained classifier dates to Alain and Bengio [1], who used probes diagnostically, to ask what information a layer carries. Safety work inverted the purpose: the probe became the deployed artifact rather than the measurement. Goldowsky-Dill et al. [10] train linear probes to flag strategic deception and report AUROC 0.96–0.999 in distribution, while cautioning that the probe may fire on the *topic* of deception rather than the intent.

The recurring finding is a gap between in-distribution and out-of-distribution behaviour. Kretschmar et al. [16] assemble a type-labelled corpus across four open-weight models and find that existing techniques systematically miss particular *kinds* of lie; Natarajan et al. [20] conclude that type-matched probes outperform any single universal one, which is a statement that the concept is not one direction. Boxo et al. [4] recover multiple deception directions by iterated null-space projection, and Yoo and Skapars [25] recast generalisation itself as selecting which principal components transfer. Our setting differs in what is held fixed: those works vary the *concept* or the *training distribution* and keep aggregation implicit, while we fix the token transform and vary only the reduction. We also adopt the evaluation standard of Parrack et al. [21]: a white-box probe should be scored by its *black-to-white boost* over the black-box monitor it would replace, not against chance.

7.2 Sparse autoencoders versus direct probes

An alternative route to safety-relevant features is unsupervised dictionary learning. Lieberum et al. [18] release sparse autoencoders across every layer of an open model family, making feature-level monitoring broadly available. Two results temper the approach. Chanin et al. [6] show *feature absorption*: under a sparsity objective a hierarchical parent feature stops firing where it should, its mass captured by more specific children, so a monitored feature can be silent exactly when it matters. More sharply, Korznikov et al. [14] find that on sparse probing and causal editing, trained dictionaries do not reliably beat random dictionaries of matched shape. The countervailing evidence is applied rather than benchmark-based: Marks et al. [19] run a blind auditing game against a model trained to conceal an objective, and dictionary-based investigation is among the techniques that recover it. The tools are therefore useful to a human investigator with time and hypotheses, which is a different claim from being reliable as an unattended monitor. It is the latter property a deployed guardrail needs.

We therefore avoid dictionary dependence entirely. The probes here read the residual stream directly, which forgoes feature-level interpretability but makes the memory and gradient analysis exact: there is no learned basis whose quality must be separately established.

7.3 AI control, ensembling, and monitoring protocols

Control protocols assume a cheap trusted monitor gating a more capable untrusted policy [11]. Gardner-Challis et al. [9] sketch a safety case for untrusted monitoring across four collusion strategies and single out passive self-recognition, noting that such protocols need monitors whose failure modes are characterised rather than assumed. Koran et al. [13] show that a diverse three-monitor ensemble beats three identical ones by $2.4\times$ at equal compute, with low error-correlation the operative property, but they do not explain what makes signals diverse.

Our cascade occupies the position these protocols reserve for a low-latency first stage and rarely characterise. The contribution is not the routing rule, which is standard, but the demonstration that its memory cost can be made independent of the adversary’s context length and that its gating is inert without calibration. Calibration follows Platt [22] and the modern treatment of Guo et al. [12]; the smooth-max operator is that of Asadi and Littman [2], imported from reinforcement learning where it was introduced to make value backups differentiable; the straight-through estimator is Bengio et al. [3].

7.4 Long-context guardrails and dilution

Kramár et al. [15] deploy activation probes in a production assistant and report the finding that motivates this paper: probes fail under short-to-long distribution shift, and *architecture choice*, not merely training data, governs whether they survive. Chen et al. [7] give the mechanism at three layers: attention mass on the unsafe span is diluted, the unsafe-versus-safe logit margin compresses proportionally, and the detection decision collapses. They measure a monotone recall drop of more than 50% across fifteen guardrails over a 0.25k–32k span, and attribute it to proportional dilution of the needle rather than to absolute length.

7.5 Activation monitors and guardrail models at deployment scale

Two lines of work bracket ours, and the contrast with each is a contrast in what is being held fixed.

Activation monitors. Rozenfeld et al. [23] monitor activations against predicate rules defined over elemental concepts, so that an operator can reconfigure safety policy without retraining a detector. Das and Fioretto [8] apply activation-based guardrails to privacy-sensitive agent traffic and argue the same economic case we do, that reading activations is far cheaper than running a second model. Both take the reduction from activations to a score as given and vary the concept vocabulary or the rule layer above it. We do the opposite, holding the concept and the token transform fixed and varying only the reduction, which is why our results are stated as a memory law and an optimisation obstruction rather than as a detection benchmark. The two are complementary: a rule engine in the style of GAVEL still needs each predicate evaluated by some reduction, and at 131,072 tokens the choice of that reduction is the difference between 8.0 MiB and 641.0 MiB per predicate (Fig. 2(a)).

Guardrail models. The deployed alternative to a probe is a dedicated classifier. Lee et al. [17] release SGuard-v1, a pair of 2B-parameter filters over a Granite-3.3-2B backbone that emit per-category risk labels and are explicitly positioned as lightweight. Lightweight is relative: a 2B forward pass is roughly four orders of magnitude more arithmetic than a probe pass over the same tokens (§6.3), and its cost is paid on every request rather than on the ambiguous fraction. That is precisely the gap a cascade is for, and it is why §6.3 instantiates stage 2 with SGuard-ContentFilter-2B rather than with a stand-in.

Protocol-aware injection. The fragmentation adversary of §6 is a protocol-aware attack in the sense of Gardner-Challis et al. [9]: it is constructed against the monitor’s aggregation rule rather than against the model’s refusal behaviour. Defences that assume the attacker writes a recognisably harmful span do not bind here, because the attacker’s budget is deliberately spread below the per-token threshold that any peak-seeking detector relies on.

7.6 What none of this addresses

Both stop short of an aggregation-controlled comparison with a memory law attached. Neither reports the optimisation obstruction of Theorem 3.1, which is invisible unless one trains a hard-max probe at weak signal, nor the fragmentation boundary of §6.1, which requires constructing the adversary that targets a maximum specifically.

7.7 Discussion, rebuttal analysis, and limitations

What we do and do not claim. Combining Props. 2.3–2.4, Rem. 3.2 and §6.1: against mean aggregation, MultiMax wins unconditionally on concentrated attacks. Against single-query softmax pooling the advantage is conditional and narrow: γ in Prop. 2.4 is a *trained* quantity, and a salient needle drives $e^\gamma \sim N/k$, cancelling the dilution; post-annealing MultiMax leads only in the weak-signal regime and ties elsewhere. Against a distributed adversary the two fail together, with MultiMax degrading less gracefully. The unconditional advantage is therefore the systems one, and the deployment recommendation follows from it: use MultiMax as a $\Theta(1)$ -memory first pass, not as a replacement for inspection.

Rebuttal analysis. Two objections shaped this paper. (i) “*Softmax outperforms MultiMax at $m=256$.*” Correct on AUROC (0.716 vs 0.523), and we report it in the abstract rather than the appendix; it is why our claim is architectural rather than a detection claim. (ii) “*Is 19.1 MiB truly $\mathcal{O}(1)$?*” Half right, and it prompted the chunk ablation of Rem. 5.1, which replaces an $\mathcal{O}(1)$ assertion with the measured law $\Theta(\min(C, N))$.

7.7.1 Deployment mechanics: bandwidth, not FLOPs

The systems argument is easy to misread as being about compute. It is not. All three $\Theta(N)$ -time aggregators perform the same $\Theta(Nmd)$ multiply-accumulates in the token transform ϕ , which dominates the FLOP count; the reduction itself is $\Theta(Nm)$ and negligible. What separates them is *traffic*. Softmax pooling writes an $(1, N, m)$ intermediate to HBM and reads it back to form the weighted sum, so it moves $\Theta(Nm)$ bytes twice; the streaming maximum writes $\Theta(H)$ bytes per chunk and never re-reads. At $N=131,072$, $m=512$ in bf16 a single such intermediate is $131,072 \times 512 \times 2\text{B} = 128\text{ MiB}$ exactly, so one write-then-read costs 256 MiB of avoidable traffic per sequence.

One clarification, since the arithmetic invites a false comparison: 128 MiB is the size of *one* intermediate, not the measured 652.1 MiB peak of Table 3. The peak additionally holds the transform output, the attention scores, the weighted product and autograd workspace, several of them live simultaneously; the bandwidth argument concerns per-tensor traffic, the memory law concerns peak residency, and they are different quantities that happen to share a cause. What the traffic figure does explain is the shape of the gap: latency differs by $1.4\times$ (12.729 vs 9.205 ms) while memory differs by $34\times$ (652.1 vs 19.1 MiB). If these kernels were arithmetic-bound the two ratios would track each other, because all three aggregators issue the same $\Theta(Nmd)$ multiply-accumulates. They do not track, which is the signature of a bandwidth-bound regime; bandwidth, not FLOPs, is what the streaming reduction buys.

Three practical consequences. *Batching.* Since overhead is $\Theta(\min(C, N))$ per sequence rather than $\Theta(N)$, admissible batch size at fixed memory is set by C , which the operator controls, rather than by the longest request in flight. *Tail latency.* The $\Theta(1)$ footprint means a 131k-token request does not evict a co-resident

model, so the guardrail does not create the pathological tail that a memory-proportional monitor does. *Choosing C .* Table 3 and Remark 5.1 make this a direct trade: $C=512$ costs 12.1 MiB, $C=4096$ costs 19.1 MiB, and larger C buys occupancy at linear memory cost until C approaches N , where the streaming property is lost entirely.

7.7.2 Adversarial countermeasures against fragmentation

§6.1 establishes that a budget-preserving adversary defeats both pooled aggregators at $m=64$. We do not have a solution, but the structure of (15) constrains what one could look like, and we state the options rather than leave the failure unaddressed.

Top- r instead of top-1. Replacing $\max_j$ by the mean of the r largest scores raises the fragmentation threshold from $m \approx 1$ to $m \approx r$ while keeping the streaming property, since a running top- r heap is $\Theta(r)$ state. This trades Theorem 2.5’s exact padding invariance for approximate invariance and is the most direct patch; it is untested here.

Ensembling across aggregators. Because mean pooling is by construction invariant to m at fixed budget ((15)), a mean-pooled member covers precisely the regime where the maximum fails. Table 8 measures this rather than assuming it: mean pooling holds AUROC 0.753–0.840 across the full $256\times$ fragmentation sweep and overtakes both peaked aggregators at $m=256$. That is error-decorrelation of the kind Koran et al. [13] observe, with an identified cause (aggregator diversity, not model diversity), which suggests it is the principled axis on which to build a monitor ensemble. The cost is reintroducing a $\Theta(N)$ member unless that member is itself streamed; a running mean is, so this particular pair is compatible with the memory law.

Budget accounting rather than peak detection. The deepest issue is that a maximum answers “is any single token anomalous” when the security question is “is the total anomalous mass large”. A detector reading accumulated deviation is the natural dual, and (15) shows it is exactly what fragmentation leaves invariant. That detector is mean pooling, whose $\Theta(1/N)$ dilution is what motivated the maximum in the first place. The two failure modes are therefore complementary rather than jointly solvable by a single pooled statistic. We regard this as the sharpest open problem the paper raises.

What we can say now. A deployment facing a fragmentation-capable adversary should not rely on a single pooled probe. The cascade framing already anticipates this: the appropriate response to $m \geq 64$ is to widen δ , escalating more traffic to full inspection, and (14) prices that decision directly.

Limitations. *Synthetic activations.* Benchmarks run on Gaussian backgrounds with a single fixed additive attack direction; real misuse features are neither isolated nor axis-aligned, and the transfer of these results to real residual streams is untested. *Known- m training.* Benchmark C trains on the same fragmentation it evaluates, which is generous to the defender; an unknown- m adversary is untested. *Threshold drift.* Rem. 2.6 predicts $\sqrt{2\log N}$ growth in the benign maximum; our protocol recalibrates per length by construction and does not characterise the drift itself. *Simulated escalation.* The cascade’s second stage is a deterministic stand-in exercising routing and cost arithmetic; it is not evidence about a real monitor, and the cost constants are illustrative. *Single scale.* One width, one head count, one GPU.

Reproducibility. Every number in this paper is read from a single JSON artifact produced by the benchmark suite, and a checker verifies each quoted figure against it. This is not incidental: three figures in an earlier draft had silently gone stale after a bug fix changed the underlying model, and the checker exists because prose is not otherwise tested.

References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.
- [2] Kavosh Asadi and Michael L. Littman. An alternative softmax operator for reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 243–252. PMLR, 2017.

- [3] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [4] Gerard Boxo, Ryan Socha, Daniel Yoo, and Shivam Raval. Caught in the act: A mechanistic approach to detecting deception. *arXiv preprint arXiv:2508.19505*, 2025.
- [5] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- [6] David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Isaac Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. In *Advances in Neural Information Processing Systems (NeurIPS 2025)*, 2025. Oral presentation.
- [7] Ziyang Chen, Xing Wu, and Songlin Hu. LongGuard: Mechanistic analysis and training-free mitigation of long-context failure in safety guardrails. *arXiv preprint arXiv:2608.27580*, 2026.
- [8] Saswat Das and Ferdinando Fioretto. NeuroFilter: Activation-based guardrails for privacy-conscious LLM agents. *arXiv preprint arXiv:2601.14660*, 2026.
- [9] Nelson Gardner-Challis, Jonathan Bostock, Georgiy Kozhevnikov, Morgan Sinclair, Joan Velja, Alessandro Abate, and Charlie Griffin. When can we trust untrusted monitoring? a safety case sketch across collusion strategies. *arXiv preprint arXiv:2602.20628*, 2026.
- [10] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. Detecting strategic deception with linear probes. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 19755–19786. PMLR, 2025. Apollo Research.
- [11] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 16295–16336. PMLR, 2024. Redwood Research.
- [12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.
- [13] Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for AI control: Diverse signals outweigh more compute. *arXiv preprint arXiv:2605.15377*, 2026.
- [14] Anton Korznikov, Andrey Galichin, Alexey Dontsov, Oleg Rogov, Ivan Oseledets, and Elena Tutubalina. Sanity checks for sparse autoencoders: Do SAEs beat random baselines? *arXiv preprint arXiv:2602.14111*, 2026.
- [15] János Kramár, Joshua Engels, Zheng Wang, Bilal Chughtai, Rohin Shah, Neel Nanda, and Arthur Conmy. Building production-ready probes for Gemini. *arXiv preprint arXiv:2601.11516*, 2026. Google DeepMind; preprint, no peer-reviewed venue as of audit.
- [16] Kieron Kretschmar, Walter Laurito, Sharan Maiya, and Samuel Marks. Liars’ bench: Evaluating lie detectors for language models. *arXiv preprint arXiv:2511.16035*, 2026. Cadenza Labs.
- [17] JoonHo Lee, HyeonMin Cho, Jaewoong Yun, Hyunjae Lee, JunKyu Lee, and Juree Seok. SGuard-v1: Safety guardrail for large language models. *arXiv preprint arXiv:2511.12497*, 2025. Samsung SDS; technical report. ContentFilter and JailbreakFilter, 2B each, Granite-3.3-2B backbone.
- [18] Tom Lieberum, Senthoran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Miami, Florida, US, 2024. Association for Computational Linguistics.

- [19] Samuel Marks, Johannes Treutlein, Trenton Bricken, Jack Lindsey, Jonathan Marcus, Siddharth Mishra-Sharma, Daniel Ziegler, Emmanuel Ameisen, Joshua Batson, Tim Belonax, Samuel R. Bowman, Shan Carter, Brian Chen, Hoagy Cunningham, Carson Denison, Florian Dietz, Satvik Golechha, Akbir Khan, Jan Kirchner, Jan Leike, Austin Meek, Kei Nishimura-Gasparian, Euan Ong, Christopher Olah, Adam Pearce, Fabien Roger, Jeanne Salle, Andy Shih, Meg Tong, Drake Thomas, Kelley Rivoire, Adam Jermy, Monte MacDiarmid, Tom Henighan, and Evan Hubinger. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025. Anthropic.
- [20] Vikram Natarajan, Devina Jain, Shivam Arora, Satvik Golechha, and Joseph Bloom. One probe won’t catch them all: Towards targeted deception detection. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*. PMLR, 2026.
- [21] Avi Parrack, Carlo Leonardo Attubato, and Stefan Heimersheim. Benchmarking deception probes via black-to-white performance boosts. *arXiv preprint arXiv:2507.12691*, 2026. Apollo Research.
- [22] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, MA, 1999.
- [23] Shir Rozenfeld, Rahul Pankajakshan, Itay Zloczower, Eyal Lenga, Gilad Gressel, and Yisroel Mirsky. GAVEL: Towards rule-based safety through activation monitoring. In *International Conference on Learning Representations (ICLR)*, 2026.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.
- [25] Daniel Yoo and Adrians Skapars. Probe generalization as subspace selection for OOD deception detection. *arXiv preprint arXiv:2609.02893*, 2026.